Taller 1 - Fundamentos de Inteligencia Artificial y Redes Neuronales

CORPORACIÓN UNIVERSITARIA DEL HUILA – CORHUILA FACULTAD DE INGENIERÍA DE SISTEMAS SEMESTRE IX 2026A

PORTADA

Taller 1: Fundamentos de Inteligencia Artificial y Redes Neuronales

Presentado por:

- Victor Andres Rojas
- Juan Sebastian Murcia
- Johan Esteban Ramirez
- Carlos Andres Villamil
- Kevin Cuesta

Asignatura: Inteligencia Artificial

Docente: José Miguel Llanos Mosquera

Institución: Corporación Universitaria del Huila – CORHUILA

Neiva, Huila

2026

TABLA DE CONTENIDO

1. Introducción
2. Desarrollo del Taller
   - 2.1. Punto 1: Fundamentos de IA y la Neurona Biológica
   - 2.2. Punto 2: El Perceptrón - Concepto, Entrenamiento y Limitaciones
   - 2.3. Punto 3: Redes Neuronales Multicapa (MLP) y Backpropagation
   - 2.4. Punto 4: Implementación Práctica del Perceptrón Simple
   - 2.5. Punto 5: Implementación de una Red Neuronal Multicapa (MLP)
3. Conclusiones
4. Referencias

1. INTRODUCCIÓN

El presente documento detalla el desarrollo integral del **Taller 1 sobre Fundamentos de Inteligencia Artificial y Redes Neuronales**. Este trabajo tiene como propósito fundamental consolidar los conocimientos adquiridos sobre los modelos de aprendizaje supervisado, desde la inspiración biológica hasta la implementación de algoritmos complejos capaces de resolver problemas no lineales.

La estructura del taller permite una transición lógica desde los conceptos abstractos de las neuronas artificiales hasta la ejecución de algoritmos en **Python**, demostrando el dominio de las estructuras básicas de las redes neuronales y su aplicación en la resolución de problemas lógicos como AND, OR y XOR.

2. DESARROLLO DEL TALLER

2.1. Punto 1: Fundamentos de Inteligencia Artificial y la Neurona Biológica

Imagina que pudieras enseñarle a una máquina no solo a seguir órdenes, sino a **aprender de la experiencia**, tal como lo hace un niño. En esencia, la Inteligencia Artificial es la rama de la informática que busca crear sistemas capaces de realizar tareas que, normalmente, requerirían de la inteligencia humana.

¿En qué se traduce esto en nuestro día a día?

La IA no es solo ciencia ficción; está en casi todo lo que tocamos:

- **Asistentes personales:** Como Siri o Alexa, que entienden nuestro lenguaje natural.
- **Recomendaciones inteligentes:** Netflix o Spotify sabiendo exactamente qué película o canción te gustará después.
- **Salud:** Algoritmos que ayudan a los médicos a detectar enfermedades en radiografías con una precisión asombrosa.
- **Vehículos autónomos:** Coches que "ven" la carretera y toman decisiones en tiempo real para evitar accidentes.

2. La Neurona Biológica: El Procesador de la Naturaleza

Para entender la IA, primero debemos mirar hacia adentro. Nuestro cerebro está compuesto por miles de millones de células especializadas llamadas **neuronas**. Son las encargadas de transmitir información mediante impulsos eléctricos y químicos.

Sus componentes principales:

1. **Dendritas (Las antenas):** Son prolongaciones ramificadas que actúan como receptores. Su función es recoger las señales (mensajes) que vienen de otras neuronas.
2. **Cuerpo Celular o Soma (El centro de control):** Aquí se procesa toda la información recibida. Si la suma de las señales es lo suficientemente fuerte, la neurona decide "disparar" un mensaje.
3. **Axón (El cable de transmisión):** Es una extensión larga por la que viaja el impulso eléctrico hacia otras células.
4. **Sinapsis (El puente):** Es el punto de comunicación. No hay contacto físico directo; las neuronas se pasan información mediante sustancias químicas (neurotransmisores).

Diagrama de la Neurona Biológica:

```
(Dendritas)
  \ | /
 -- O -- (Cuerpo Celular / Soma)
    |
    | (Axón)
    |
  / | \
(Sinapsis / Terminales)
```

3. Del Cerebro al Código: El Perceptrón Artificial

El **perceptrón** es la unidad básica de las redes neuronales artificiales. Fue diseñado como un intento de imitar matemáticamente lo que hace una sola neurona biológica.

La Analogía: ¿Cómo se modela lo natural en lo digital?

Componente Biológico	Componente del Perceptrón	Explicación
Dendritas	**Entradas (x_n)**	Los datos o señales que recibe el sistema.
Sinapsis	**Pesos (w_n)**	En el cerebro, algunas conexiones son más fuertes que otras. En la IA, los "pesos" determinan qué tan importante es una entrada específica.
Cuerpo Celular	**Función de Suma y Activación**	El perceptrón suma las entradas multiplicadas por sus pesos. Si el resultado supera un "umbral", genera una salida (igual que la neurona biológica decide si disparar).
Axón	**Salida (y)**	El resultado final del procesamiento que se envía a la siguiente capa o como respuesta final.

4. Diagrama Comparativo

A continuación, vemos cómo se visualiza el modelo computacional inspirado en la biología:

Estructura del Perceptrón Artificial:

$$Entradas(x) \rightarrow [Pesos(w)] \rightarrow \sum(Suma) \rightarrow \int(Activación) \rightarrow Salida(y)$$

1. **Entradas ($x_1, x_2, \cdots$):** La información que llega (ej. los píxeles de una imagen).
2. **Pesos ($w_1, w_2, \cdots$):** El grado de importancia que le damos a cada dato.
3. **Suma Producida:** Se calcula como $\sum(x_i \cdot w_i)$.
4. **Función de Activación:** El filtro que decide si la información es relevante para pasar a la siguiente etapa.

Conclusión

La Inteligencia Artificial no es más que un espejo, un tanto simplificado, pero increíblemente potente, de nuestra propia biología. Al entender cómo una neurona procesa información, entendemos los cimientos sobre los que se construyen los robots y algoritmos más avanzados del mundo actual. Pasamos de impulsos químicos a impulsos eléctricos en un chip, pero el objetivo sigue siendo el mismo: **procesar, aprender y decidir.**

2.2. Punto 2: El Perceptrón - Concepto, Entrenamiento y Limitaciones

1. Introducción

El Perceptrón es uno de los modelos base en el desarrollo de las redes neuronales. Su importancia radica en que introduce la idea de aprender pesos (w) y un sesgo (b) para tomar decisiones de clasificación. En este documento se presenta su modelo matemático, su algoritmo de entrenamiento y sus limitaciones, con énfasis en la razón matemática por la cual no puede resolver problemas no lineales como XOR. Además, se incluye una implementación en Python (NumPy) con explicación línea por línea y los resultados esperados.

2. Definición del Perceptrón y modelo matemático (con función de activación)

El Perceptrón es un clasificador binario que asigna una entrada $x = (x1, x2, \ldots, xn)$ a una clase $\hat{y} \in \{0,1\}$. Primero calcula una combinación lineal y luego aplica una función de activación.
Modelo matemático:
1) Suma ponderada (combinación lineal):
$z = \Sigma (w_i \cdot x_i) + b$ (equivalente: $z = w^T x + b$)
2) Predicción (salida):
$\hat{y} = f(z)$
Función de activación típica del Perceptrón (escalón/Heaviside):
$f(z) = 1$ si $z \geq 0$
$f(z) = 0$ si $z < 0$
Interpretación geométrica: la frontera de decisión está dada por $w^T x + b = 0$. En 2D es una recta; por lo tanto, el Perceptrón solo puede separar clases si existe una separación lineal.

2.1 Las fórmulas del Perceptrón en Python (cómo se ven en código)

Equivalente directo de $z = w^T x + b$ y $\hat{y} = f(z)$ en Python:

```
z = np.dot(w, x) + b  # z = w·x + b
y_hat = 1 if z >= 0 else 0  # función escalón
```

3. Algoritmo de entrenamiento del Perceptrón (ajuste de pesos y bias)

El Perceptrón aprende corrigiendo errores. Para cada ejemplo (x, y), calcula una predicción $\hat{y}$. Si se equivoca, ajusta pesos y bias para aumentar la probabilidad de clasificar correctamente ese ejemplo la próxima vez.
Pasos (para un ejemplo):
1) $\hat{y} = f(w^T x + b)$
2) $error = y - \hat{y}$
3) Actualización de pesos: $w \leftarrow w + \eta \cdot error \cdot x$
4) Actualización del sesgo: $b \leftarrow b + \eta \cdot error$
Donde η (eta) es el learning rate. Si $error = 0$, no hay cambio. Si $error \neq 0$, el ajuste mueve la frontera para corregir la clasificación.

3.1 Regla de actualización en Python (forma directa)

```
error = y - y_hat
w = w + lr * error * x  # actualizar pesos
```

b = b + lr * error # actualizar bias

4. Ejemplos: problemas que el Perceptrón puede y no puede resolver (con justificación matemática)

4.1 Ejemplo que SÍ puede: AND (linealmente separable)

Tabla AND: (0,0)→0, (0,1)→0, (1,0)→0, (1,1)→1.
Una solución posible es: $w = [1, 1]$ y $b = -1.5$. Verificación sustituyendo en $z = w1 \cdot x1 + w2 \cdot x2 + b$:
• (0,0): $z = 0 + 0 - 1.5 = -1.5 < 0 \Rightarrow \hat{y} = 0$
• (0,1): $z = 0 + 1 - 1.5 = -0.5 < 0 \Rightarrow \hat{y} = 0$
• (1,0): $z = 1 + 0 - 1.5 = -0.5 < 0 \Rightarrow \hat{y} = 0$
• (1,1): $z = 1 + 1 - 1.5 = 0.5 \geq 0 \Rightarrow \hat{y} = 1$
Como existe una recta que separa las clases, AND es linealmente separable y el Perceptrón puede aprenderlo.

4.2 Ejemplos que NO puede: XOR y fronteras no lineales

Tabla XOR: (0,0)→0, (0,1)→1, (1,0)→1, (1,1)→0.
XOR no es linealmente separable: no existe una sola recta que deje juntos los 1 y separados los 0. Esto se puede demostrar con desigualdades.

5. Limitación principal: demostración matemática de por qué el Perceptrón NO puede resolver XOR

Supongamos que existe un Perceptrón con pesos w1, w2 y bias b que resuelve XOR.
El Perceptrón predice 1 si $w1 \cdot x1 + w2 \cdot x2 + b \geq 0$, y predice 0 si $w1 \cdot x1 + w2 \cdot x2 + b < 0$.
Entonces deberían cumplirse estas condiciones:
1) Para (0,0)→0: $b < 0$
2) Para (1,1)→0: $w1 + w2 + b < 0$
3) Para (1,0)→1: $w1 + b \geq 0$
4) Para (0,1)→1: $w2 + b \geq 0$
Ahora sumamos (3) y (4):
$$(w1 + b) + (w2 + b) \geq 0 \Rightarrow w1 + w2 + 2b \geq 0$$

Pero (2) exige: w1 + w2 + b < 0. Si además (1) dice que b < 0, entonces 2b < b. Eso hace imposible que w1 + w2 + 2b sea ≥ 0 mientras w1 + w2 + b sea < 0. Contradicción. Por lo tanto, no existe (w1, w2, b) que resuelva XOR con un Perceptrón simple.
Conclusión: como XOR requiere una frontera no lineal (o la combinación de varias rectas), un único Perceptrón no puede resolverlo. Por eso se necesitan redes multicapa (MLP).

6. Implementación en Python (NumPy) con explicación en cada línea

El siguiente código entrena un Perceptrón para un problema SÍ linealmente separable (AND). Se incluyen comentarios en cada línea para explicar qué hace y para qué sirve. Nota: Si se reemplaza AND por XOR, el modelo no logra clasificar correctamente los 4 casos a la vez.

```
import numpy as np  # Usamos NumPy para arreglos (vectores/matrices) y operaciones como np.dot

def step(z):  # Función escalón (activación): convierte z en clase 0 o 1
    return 1 if z >= 0 else 0  # Si z >= 0 clasifica como 1; si no, como 0

# ---------- Dataset AND (linealmente separable) ----------
X = np.array([[0,0], [0,1], [1,0], [1,1]])  # 4 ejemplos, 2 entradas por ejemplo
y = np.array([0,0,0,1])  # Etiquetas AND: solo es 1 cuando (1,1)

# ---------- Hiperparámetros ----------
lr = 0.1  # Learning rate: tamaño del ajuste en cada actualización
epochs = 20  # Número de épocas: cuántas veces recorremos todo el dataset

# ---------- Inicialización ----------
w = np.zeros(X.shape[1])  # Pesos iniciales en 0. X.shape[1]=2 entradas ⇒ w tiene tamaño 2
b = 0.0  # Bias inicial (umbral)

# ---------- Entrenamiento ----------
for epoch in range(epochs):  # Repetimos el aprendizaje varias épocas
    for i in range(len(X)):  # Recorremos ejemplo por ejemplo
        x = X[i]  # Tomamos el ejemplo i (vector con 2 valores)
        z = np.dot(w, x) + b  # Fórmula del perceptrón: z = w·x + b
        y_hat = step(z)  # Predicción: aplica función escalón para obtener 0 o 1
        error = y[i] - y_hat  # Error: 0 si acierta; ±1 si se equivoca
        w = w + lr * error * x  # Actualiza pesos: mueve la frontera según el ejemplo x
        b = b + lr * error  # Actualiza bias: ajusta el umbral de decisión

# ---------- Resultados ----------
print('Pesos finales:', w)  # Muestra w aprendido
```

```
print('Bias final:', b)  # Muestra b aprendido

print('\nPredicciones en AND:')  # Título para ver resultados
for i in range(len(X)):  # Probamos el modelo con los mismos ejemplos
    z = np.dot(w, X[i]) + b  # Calcula z para el ejemplo i
    print(X[i], '->', step(z))  # Muestra entrada y salida predicha
```

6.1 ¿Qué se espera al final al ejecutar el código?

Resultado esperado (conceptual): el Perceptrón debe clasificar correctamente el operador AND. Por lo tanto, al imprimir las predicciones se espera ver:

```
[0 0] -> 0  # AND(0,0)=0
[0 1] -> 0  # AND(0,1)=0
[1 0] -> 0  # AND(1,0)=0
[1 1] -> 1  # AND(1,1)=1
```

Los valores exactos de w y b pueden variar según inicialización y número de épocas, pero mientras las predicciones sean correctas, el entrenamiento fue exitoso.

6.2 ¿Qué pasaría si intento entrenar XOR con Perceptrón?

Si se reemplaza y por XOR (y = [0,1,1,0]), el Perceptrón no logra una solución estable: puede acertar algunos casos, pero necesariamente fallará al menos uno, porque no existe una frontera lineal que separe XOR (demostrado en la sección 5).

7. Conclusión

El Perceptrón es un modelo clave para comprender el aprendizaje supervisado en redes neuronales. Su entrenamiento ajusta pesos y bias para construir una frontera lineal. Sin embargo, su limitación principal es que solo resuelve problemas linealmente separables. El caso XOR demuestra matemáticamente esta restricción, lo cual motivó la creación de modelos más potentes como las Redes Neuronales Multicapa (MLP), capaces de aprender fronteras no lineales. Lo que NO usamos en el Perceptrón simple son las Derivadas, Funciones sigmoid, ReLU, Gradientes continuos, Eso pertenece a MLP y Backpropagation.

2.3. Punto 3: Redes Neuronales Multicapa (MLP) y el Algoritmo de Backpropagation

3.1 Estructura de una Red Neuronal Multicapa (MLP)

Una Red Neuronal Multicapa (MLP – Multilayer Perceptron) es una extensión del perceptrón simple propuesto por Frank Rosenblatt.
Su principal diferencia es que incorpora una o más capas ocultas, lo que le permite modelar relaciones no lineales.
Una MLP está compuesta por tres tipos de capas:

3.1.1 Capa de Entrada

- Recibe las variables del problema (features).
- Cada neurona representa una característica.
- No realiza procesamiento complejo.
- Solo distribuye la información hacia la siguiente capa.

Ejemplo:
Si el problema tiene 2 variables (x1 y x2), habrá 2 neuronas de entrada.

3.1.2 Capas Ocultas

- Son el núcleo del aprendizaje.
- Cada neurona realiza:

$$z = \sum w_i x_i + b$$
$$a = f(z)$$

Donde:
w_i = pesos
b = sesgo
$f(z)$ = función de activación
Las capas ocultas permiten transformar el espacio de entrada en representaciones intermedias más complejas.

3.1.3 Capa de Salida

- Produce el resultado final.
- Su estructura depende del problema:
- 1 neurona → Clasificación binaria
- n neuronas → Clasificación multiclase
- 1 neurona continua → Regresión

Representación General MLP
Entrada → Capas Ocultas → Salida
Cada conexión tiene un peso ajustable que se optimiza durante el entrenamiento.

3.2 Rol de las Funciones de Activación No Lineales

Las funciones de activación permiten que la red modele relaciones no lineales. Si solo se utilizaran funciones lineales, múltiples capas equivaldrían a una sola transformación lineal. Las más utilizadas son:

3.2.1 Sigmoid

$$\boldsymbol{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

- Rango: (0,1)
- Usada en clasificación binaria
- Problema: desvanecimiento del gradiente

3.2.2 Tanh

$$tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$

- Rango: (-1,1)
- Centrada en cero
- Mejor convergencia que sigmoid

3.2.3 ReLU (Rectified Linear Unit)

$$ReLU(z) = max(0, z)$$

- Computacionalmente eficiente
- Reduce el problema de gradientes pequeños
- Muy usada en redes profundas

Sin funciones no lineales:

- No se pueden aprender fronteras de decisión complejas.
- No se puede resolver el problema XOR.
- La red pierde capacidad expresiva.

3.3 Algoritmo de Backpropagation

Ajustar los pesos de la red para minimizar el error entre la salida predicha y la salida real.

3.3.1 Paso 1: Propagación hacia adelante (Forward Pass)

Se calculan las activaciones capa por capa hasta obtener la salida.

3.3.2 Paso 2: Cálculo del Error

Ejemplo con error cuadrático medio:

$$E = \frac{1}{2}(y_{real} - y_{pred})^2$$

3.3.3 Paso 3: Cálculo de Gradientes

Se aplica la regla de la cadena para calcular:

$$\frac{\partial w}{\partial E}$$

Esto mide cuánto afecta cada peso al error total.

3.3.4 Paso 4: Actualización de Pesos

$$w_{nuevo} = w_{viejo} - \eta \frac{\partial w}{\partial E}$$

Donde:

- η = tasa de aprendizaje.

El error se propaga desde la salida hacia atrás por eso se llama Backpropagation.

3.4 Ventajas del MLP sobre el Perceptrón Simple

Característica	Perceptrón	MLP
Capas	1	Varias
Linealidad	Solo lineal	No lineal
Resuelve XOR	No	Si

Capacidad de modelado	Limitada	Alta

El perceptrón simple solo funciona si los datos son linealmente separables.
El MLP puede aproximar cualquier función continua (Teorema de Aproximación Universal).

3.5 Ejemplo Paso a Paso: Resolución del Problema XOR

3.5.1 Definición

x1	x2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

3.5.2 Arquitectura Necesaria

- 2 neuronas de entrada
- 2 neuronas en capa oculta
- 1 neurona de salida

3.5.3 Paso 1: Inicialización

Se asignan pesos aleatorios pequeños.

3.5.4 Paso 2: Forward Pass

Ejemplo con entrada (1,0):
Se calcula:

$$Z_1 = w_{11}x_1 + w_{12}x_2 + b_1$$
$$a_1 = f(z_1)$$

Se hace lo mismo para la segunda neurona oculta y luego para la salida.

3.5.5 Paso 3: Cálculo del Error

Se compara salida obtenida con salida esperada.

3.5.6 Paso 4: Backpropagation

- Se calcula el gradiente en la salida.
- Se propaga el error hacia la capa oculta.
- Se actualizan los pesos.

3.5.7 Resultado Final

Después de múltiples iteraciones:
La red aprende internamente:

- Una neurona oculta aproxima OR
- Otra aproxima AND
- La salida combina ambas

$$XOR = OR - AND$$

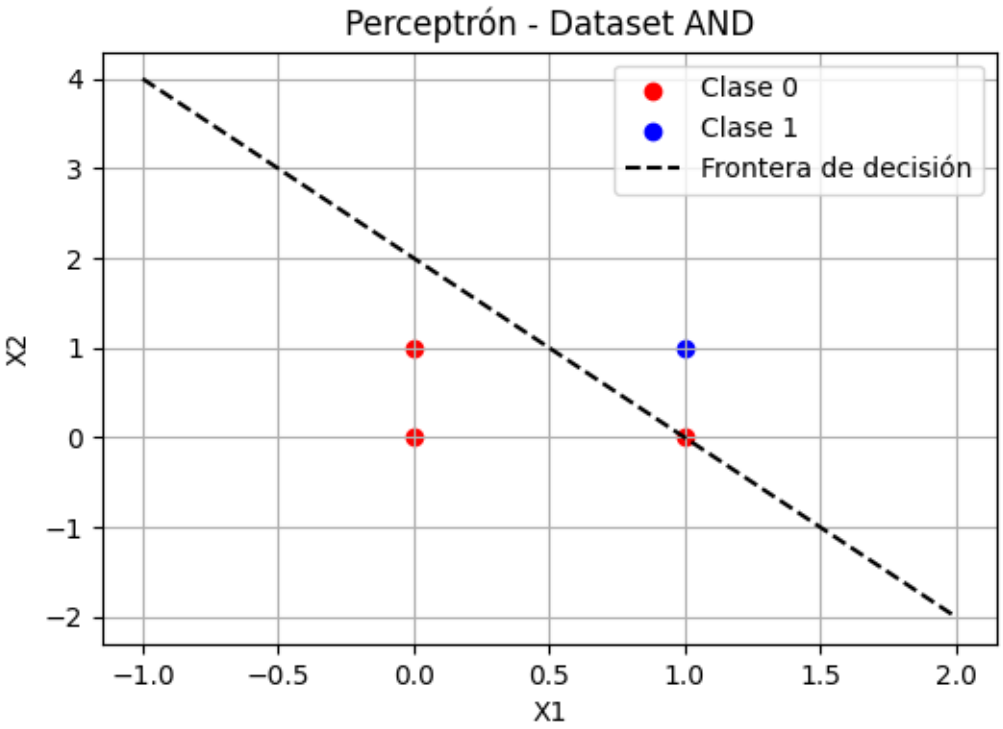
Finalmente clasifica correctamente las 4 combinaciones.

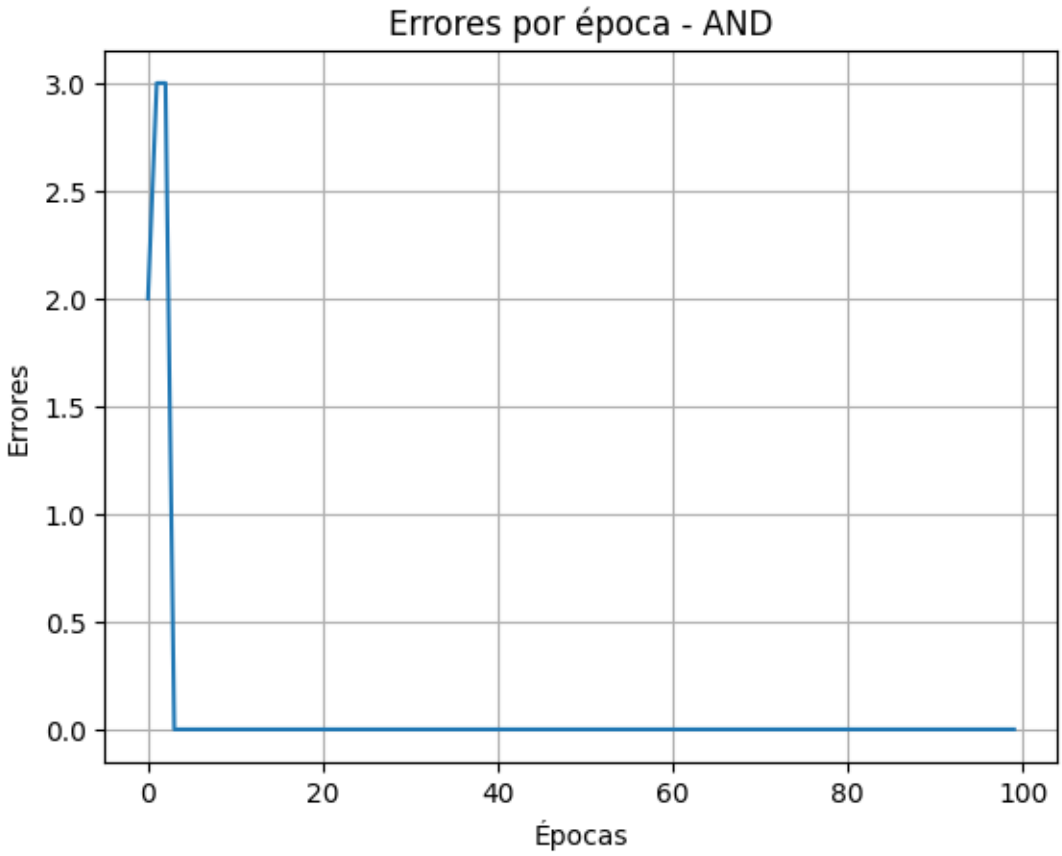
2.4. Punto 4: Implementación de un Perceptrón Simple

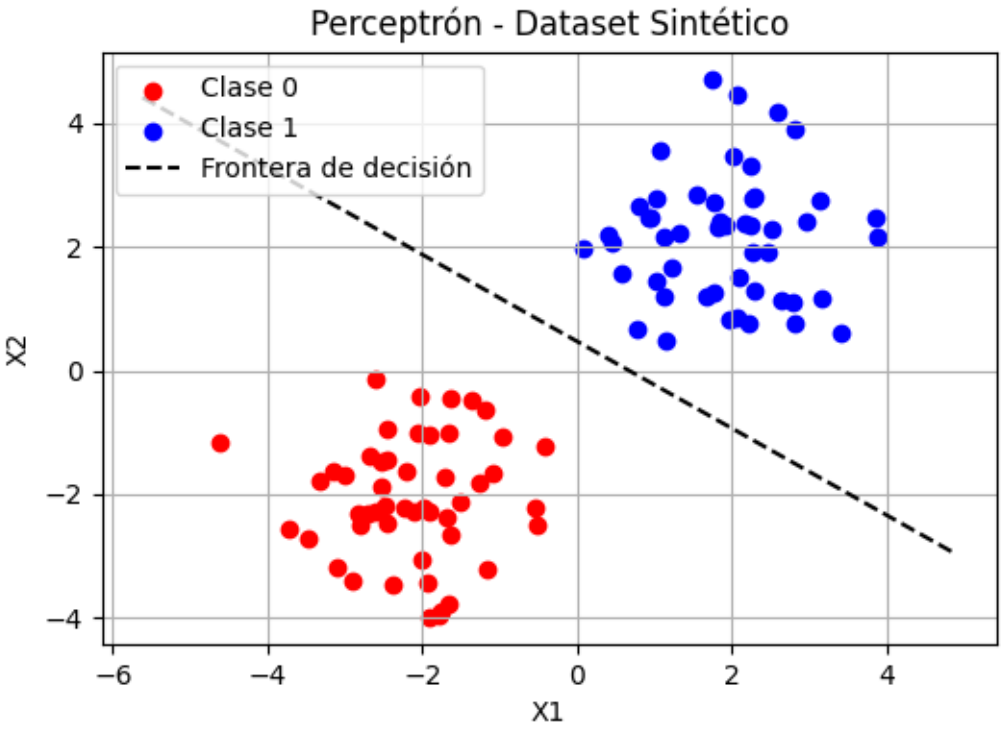
Se implementó un perceptrón desde cero para resolver problemas linealmente separables (AND y Datos Sintéticos).

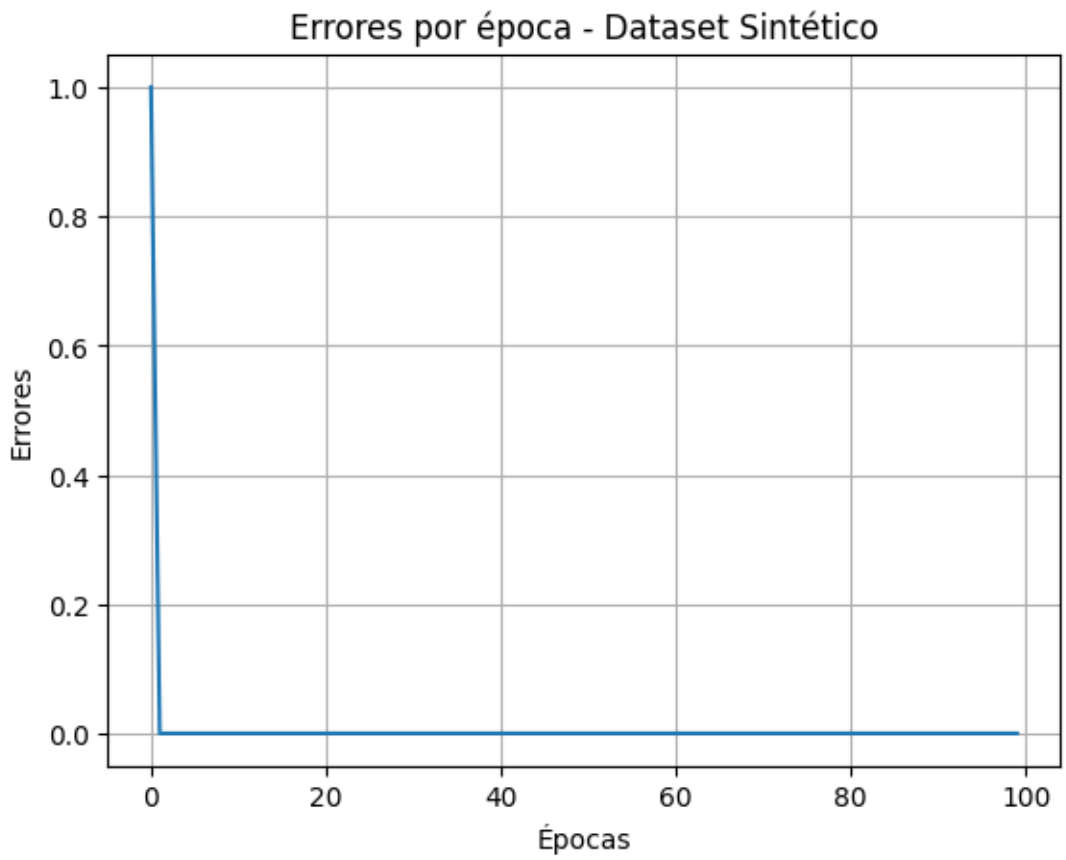
```
import numpy as np
import matplotlib.pyplot as plt

class Perceptron:
    def __init__(self, learning_rate=0.1, epochs=100):
        self.lr = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = None
        self.errors = []

    def activation_function(self, x):
        return np.where(x >= 0, 1, 0)

    def fit(self, X, y):
        self.weights = np.zeros(X.shape[1])
        self.bias = 0
```

```
        for epoch in range(self.epochs):
            error_count = 0
            for xi, target in zip(X, y):
                linear_output = np.dot(xi, self.weights) + self.bias
                y_pred = self.activation_function(linear_output)
                update = self.lr * (target - y_pred)
                self.weights += update * xi
                self.bias += update
                if update != 0: error_count += 1
            self.errors.append(error_count)
            if error_count == 0: break

    def predict(self, X):
        return self.activation_function(np.dot(X, self.weights) + self.bias)

# Prueba con AND
X_and = np.array([[0,0], [0,1], [1,0], [1,1]])
y_and = np.array([0, 0, 0, 1])
p = Perceptron(epochs=10)
p.fit(X_and, y_and)
print("Predicciones AND:", p.predict(X_and))
```


Perceptrón - Dataset AND
Clase 0
Clase 1
Frontera de decisión
X1
X2
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
4
3
2
1
0
−1
−2


Errores por época - AND
Errores
Épocas
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0
20
40
60
80
100


Perceptrón - Dataset Sintético
Clase 0
Clase 1
Frontera de decisión
X1
X2
−6
−4
−2
0
2
4
−4
−2
0
2
4



2.5. Punto 5: Implementación de una Red Neuronal Multicapa (MLP) con Backpropagation

Para resolver el problema XOR, se implementó una red con una capa oculta.

import numpy as np

import matplotlib.pyplot as plt

```
# -------------------------
# Funciones de activación
# -------------------------
def sigmoid(x):
    return 1 / (1 + np.exp(-x))


def sigmoid_derivative(x):
    return x * (1 - x)


# -------------------------
# Clase MLP
# -------------------------
class MLP:
    def __init__(self, input_size, hidden_layers, output_size, learning_rate=0.1):
        self.learning_rate = learning_rate


        # Arquitectura completa
        layers = [input_size] + hidden_layers + [output_size]
```

```
    # Inicialización de pesos y bias
    self.weights = []
    self.biases = []

    for i in range(len(layers) - 1):
        self.weights.append(np.random.randn(layers[i], layers[i + 1]))
        self.biases.append(np.zeros((1, layers[i + 1])))

# -------------------------
# Forward Propagation
# -------------------------
def forward(self, X):
    activations = [X]

    for w, b in zip(self.weights, self.biases):
        net = np.dot(activations[-1], w) + b
        activations.append(sigmoid(net))

    return activations
```

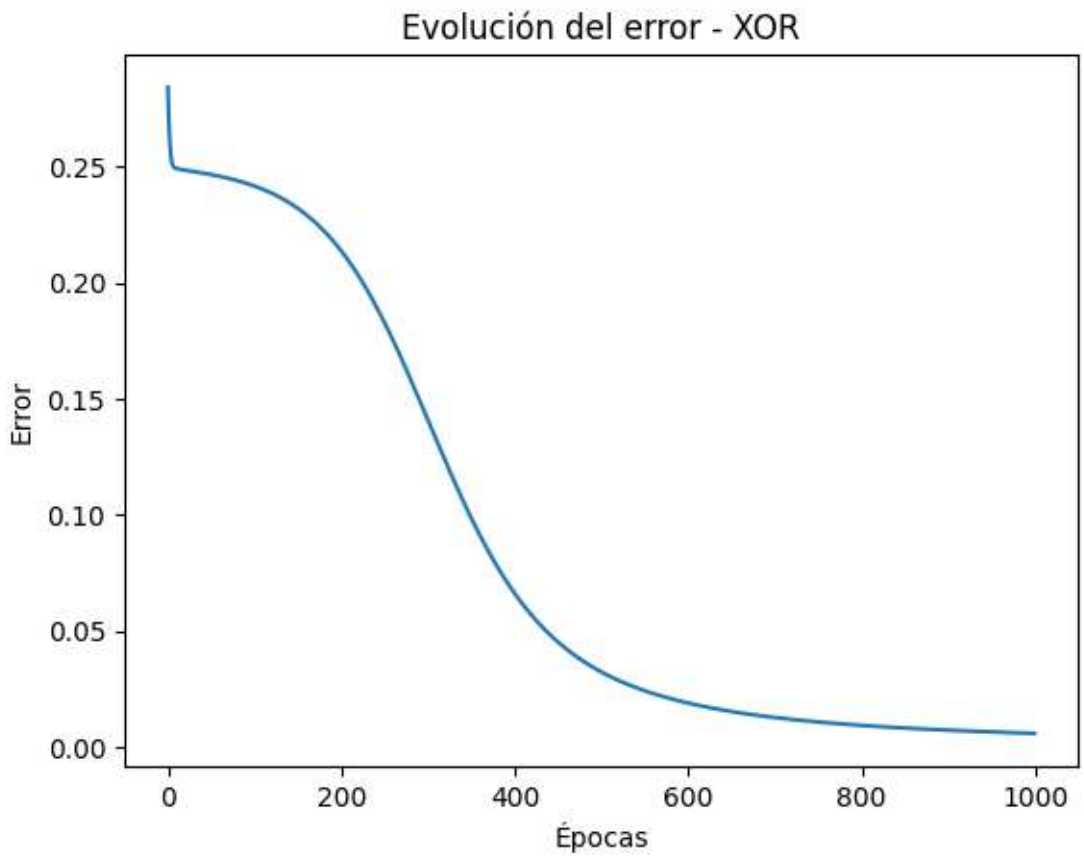
```
# -------------------------
# Backpropagation
# -------------------------
def backward(self, X, y, activations):
    error = y - activations[-1]
    deltas = [error * sigmoid_derivative(activations[-1])]

    # Propagación del error hacia atrás
    for i in reversed(range(len(self.weights) - 1)):
        delta = deltas[-1].dot(self.weights[i + 1].T) * sigmoid_derivative(activations[i + 1])
        deltas.append(delta)

    deltas.reverse()

    # Actualización de pesos y bias
    for i in range(len(self.weights)):
        self.weights[i] += activations[i].T.dot(deltas[i]) * self.learning_rate
        self.biases[i] += np.sum(deltas[i], axis=0, keepdims=True) * self.learning_rate
```

```
# -------------------------
# Entrenamiento
# -------------------------
def train(self, X, y, epochs=1000):
    errors = []

    for epoch in range(epochs):
        activations = self.forward(X)
        self.backward(X, y, activations)

        loss = np.mean((y - activations[-1]) ** 2)
        errors.append(loss)

        if epoch % 100 == 0:
            print(f"Época {epoch} - Error: {loss:.4f}")

    return errors
```

```
    # -------------------------
    # Predicción
    # -------------------------
    def predict(self, X):
        output = self.forward(X)[-1]
        return (output > 0.5).astype(int)


# -------------------------
# Dataset XOR
# -------------------------
X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])


y = np.array([
    [0],
```

```
    [1],
    [1],
    [0]
])

# -------------------------
# Creación y entrenamiento
# -------------------------
mlp = MLP(
    input_size=2,
    hidden_layers=[4],   # 1 capa oculta con 4 neuronas
    output_size=1,
    learning_rate=0.5
)

errors = mlp.train(X, y, epochs=1000)

# -------------------------
# Resultados
```

```
# -------------------------
print("\nPredicciones finales:")
print(mlp.predict(X))
# -------------------------
# Gráfica del error
# -------------------------
plt.plot(errors)
plt.xlabel("Épocas")
plt.ylabel("Error")
plt.title("Evolución del error - XOR")
plt.show()
```



3. CONCLUSIONES

1. **Fundamentos Biológicos:** Se comprendió que el diseño de las redes neuronales artificiales es una abstracción de los procesos de las neuronas biológicas.
2. **Limitación Lineal:** Se demostró que el Perceptrón Simple no puede resolver problemas no lineales como XOR.
3. **Poder del MLP:** La incorporación de capas ocultas permite resolver problemas complejos mediante el teorema de aproximación universal.
4. **Backpropagation:** Este algoritmo es esencial para ajustar los pesos minimizando el error a través de múltiples capas.

5. **Activación:** La elección de funciones como ReLU y Sigmoid es determinante para el éxito del entrenamiento.

4. REFERENCIAS

- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- Rosenblatt, F. (1958). *The Perceptron Model*. Psychological Review.
- Rumelhart, D. E. (1986). *Learning representations by back-propagating errors*. Nature.
- Material de clase, Bloques 2 y 3.